



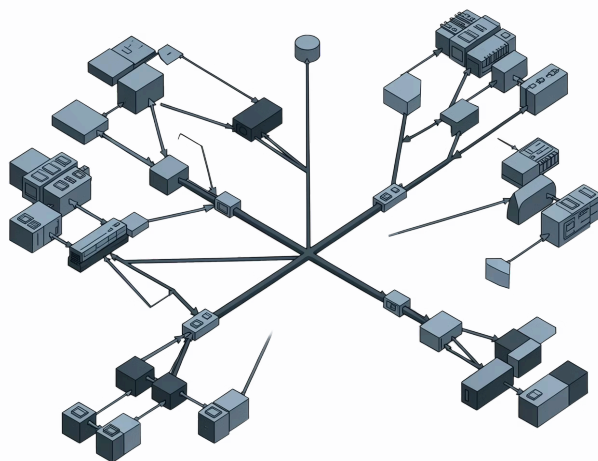
Inspecting Components & Testing with Jest

A practical guide to debugging and testing for React Native developers.



Why Debugging Matters

- ☐ Your app looks wrong or doesn't work
Something is off — and you need to find out why, fast.
- ☐ Inspect, don't guess
Use tools to see what's actually happening inside your components.
- ☐ Example: Button doesn't respond
Check if `disabled=true` is accidentally set — one prop, one fix.



Understanding Component Hierarchy

React Native apps are **trees of nested components**. Every screen is built from parent and child relationships.

A login screen looks like this:

- App → View → Text, TextInput, Button

❏ Layout problems often come from a **parent** component, not the child you're staring at.

What to Inspect: Props

Props = data passed from parent to child

They control how a component looks and behaves from the outside.

Example

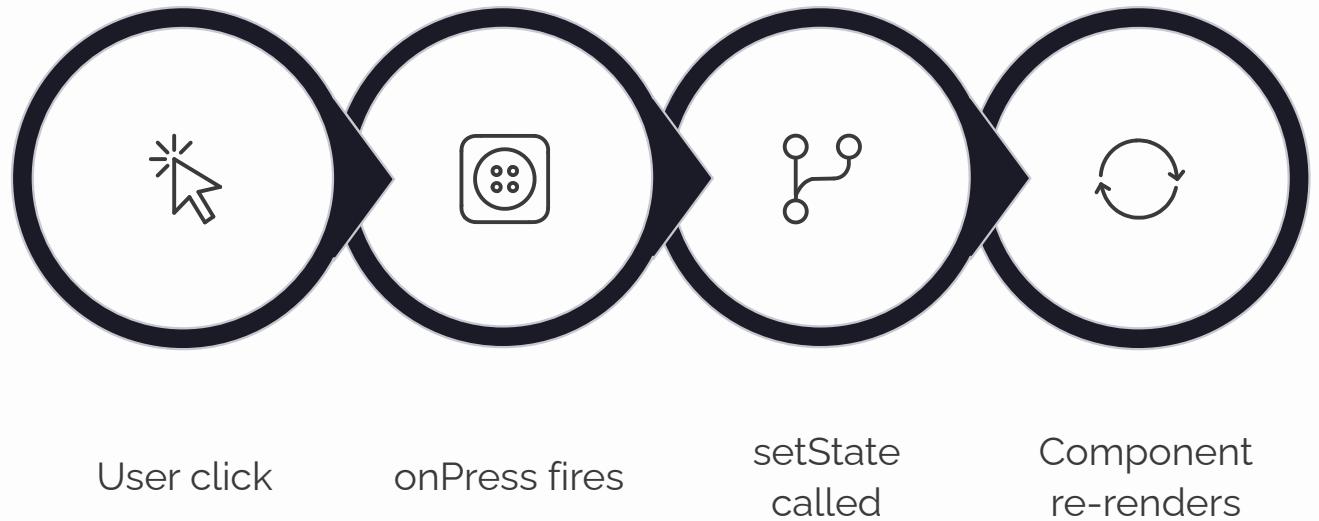
```
<Button title="Login" disabled={false} />
```

The key question to ask
Is the child receiving the **correct information** from its parent?

What to Inspect: State

State = data that changes while your app is running. When state changes, the component re-renders.

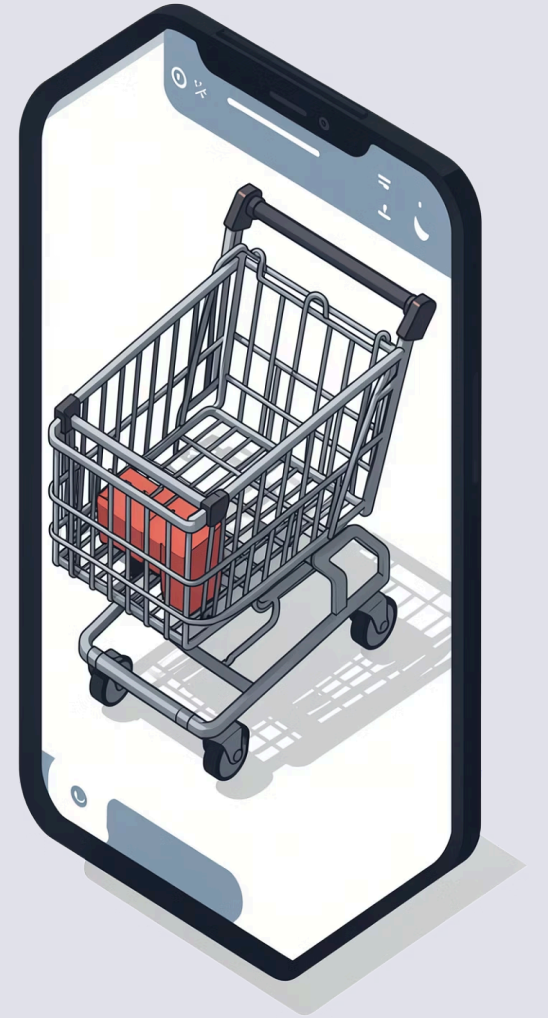
If your UI isn't updating, trace this flow step by step with `console.log()`:



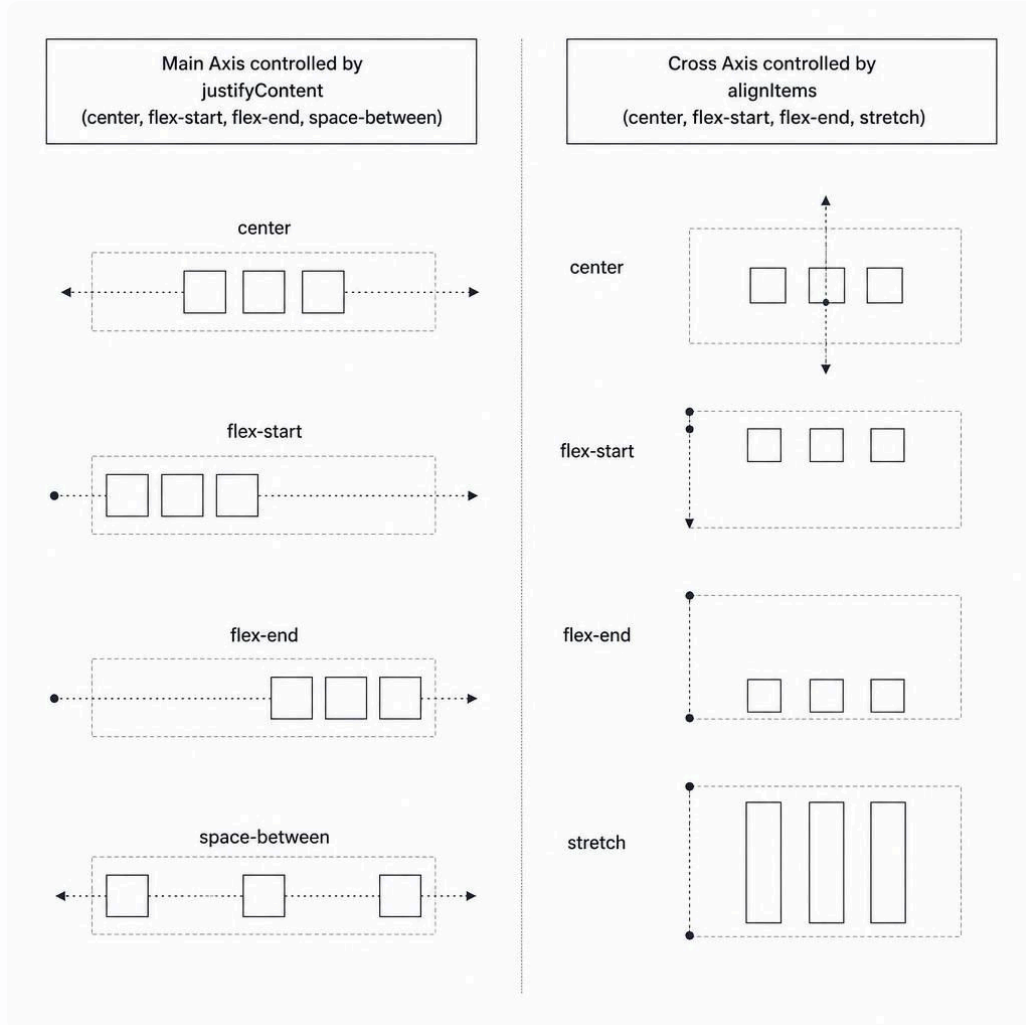
Example: Shopping Cart Bug



Cart count stays at 0? Log each step above to find exactly where the chain breaks.



What to Inspect: Styles



Layout problems almost always trace back to these three properties:

- **flex** — how much space a component takes
- **`justifyContent`** — positions items along the **main axis**
- **`alignItems`** — positions items along the **cross axis**

When something looks off, check the parent container's flex settings first.

Component Inspection Workflow



Identify the broken component

e.g. Button doesn't respond to taps



Check props & onPress handler

Is the component receiving the right data? Is the handler connected?



Trace function logic & state

Does the function run? Does state update? Does the UI re-render?



Check API calls if needed

Is the network response what you expected?



Systematic inspection finds the real problem **faster** than random guessing every time.



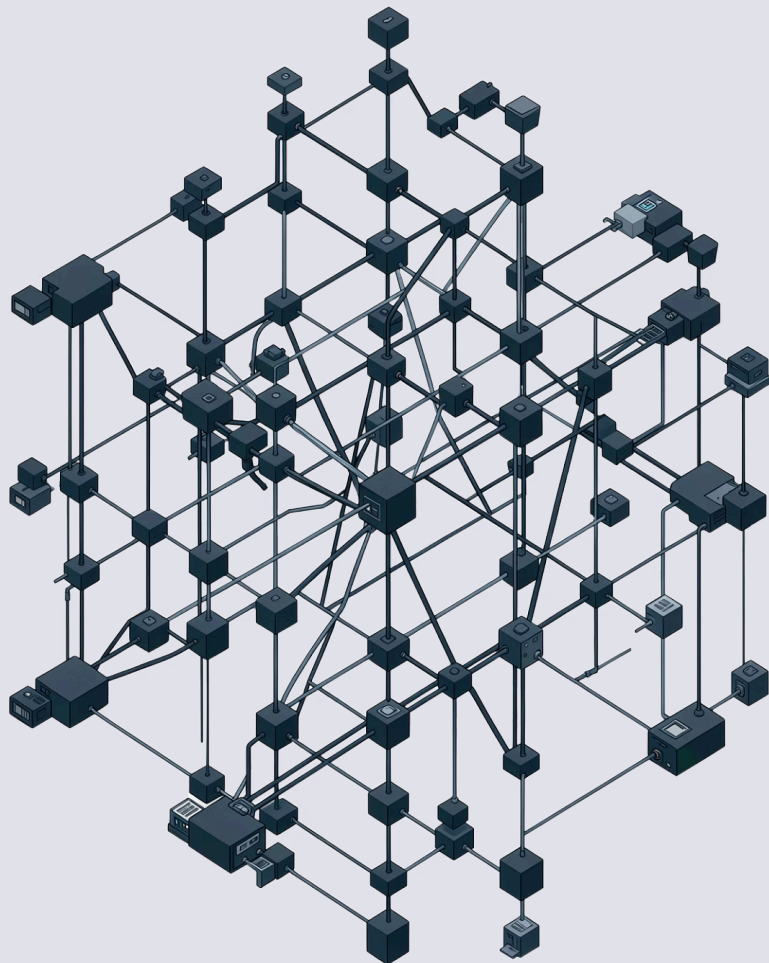
Introduction to Testing

Testing means automatically checking whether your code behaves exactly as expected — every single time.

A classic example:

A calculator function should return **5** when adding $2 + 3$. Write a test, and Jest will verify this for you automatically.

Tests catch bugs **before your users find them** — saving time, embarrassment, and bad reviews.



Why Testing Matters at Scale



Many interconnected features

Large apps have Cart, Login, Checkout, Profile — all talking to each other.



One change, unexpected breakage

Changing Cart code might accidentally break Login. It happens more than you'd think.



Tests catch regressions instantly

Automated tests run in seconds and flag breakage before it ships.

Three Levels of Testing

1

Unit Testing

Test one function in isolation – e.g., `add(2, 3)` returns 5

2

Integration Testing

Test multiple parts working together – e.g., Cart + Checkout flow

3

End-to-End Testing

Test complete user workflows – e.g., sign up → browse → purchase

Start with unit tests, then build up. Each level adds confidence that your app works in the real world.